JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface MemoryMXBean

All Superinterfaces:

`PlatformManagedObject`

```
public interface MemoryMXBean
extends PlatformManagedObject
```

The management interface for the memory system of the Java virtual machine.

A Java virtual machine has a single instance of the implementation class of this interface. This instance implementing this interface is an MXBean that can be obtained by calling the `ManagementFactory.getMemoryMXBean()` method or from the `platform MBeanServer` method.

The `ObjectName` for uniquely identifying the MXBean for the memory system within an MBeanServer is:

```
java.lang:type=Memory
```

It can be obtained by calling the `PlatformManagedObject.getObjectName()` method.

Memory

The memory system of the Java virtual machine manages the following kinds of memory:

1. Heap

The Java virtual machine has a *heap* that is the runtime data area from which memory for all class instances and arrays are allocated. It is created at the Java virtual machine start-up. Heap memory for objects is reclaimed by an automatic memory management system which is known as a *garbage collector*.

The heap may be of a fixed size or may be expanded and shrunk. The memory for the heap does not need to be contiguous.

2. Non-Heap Memory

The Java virtual machine manages memory other than the heap (referred as *non-heap memory*).

The Java virtual machine has a *method area* that is shared among all threads. The method area belongs to non-heap memory. It stores per-class structures such as a runtime constant pool, field and method data, and the code for methods and constructors. It is created at the Java virtual machine start-up.

The method area is logically part of the heap but a Java virtual machine implementation may choose not to either garbage collect or compact it. Similar to the heap, the method area may be of a fixed size or may be expanded and shrunk. The memory for the method area does not need to be contiguous.

In addition to the method area, a Java virtual machine implementation may require memory for internal processing or optimization which also belongs to non-heap memory. For example, the JIT compiler requires memory for storing the native machine code translated from the Java virtual machine code for high performance.

Memory Pools and Memory Managers

`Memory pools` and `memory managers` are the abstract entities that monitor and manage the memory system of the Java virtual machine.

A memory pool represents a memory area that the Java virtual machine manages. The Java virtual machine has at least one memory pool and it may create or remove memory pools during execution. A memory pool can belong to either the heap or the non-heap memory.

A memory manager is responsible for managing one or more memory pools. The garbage collector is one type of memory manager responsible for reclaiming memory occupied by unreachable objects. A Java virtual machine may have one or more memory managers. It may add or remove memory managers during execution. A memory pool can be managed by more than one memory manager.

Memory Usage Monitoring

Memory usage is a very important monitoring attribute for the memory system. The memory usage, for example, could indicate:

- the memory usage of an application,
- the workload being imposed on the automatic memory management system,
- potential memory leakage.

The memory usage can be monitored in three ways:

- Polling
- Usage Threshold Notification
- Collection Usage Threshold Notification

Details are specified in the `MemoryPoolMXBean` interface.

The memory usage monitoring mechanism is intended for load-balancing or workload distribution use. For example, an application would stop receiving

any new workload when its memory usage exceeds a certain threshold. It is not intended
for an application to detect and recover from a low memory condition.

Notifications

This `MemoryMXBean` is a `NotificationEmitter` that emits two types of memory `notifications` if any one of the memory pools supports a usage threshold or a collection usage threshold which can be determined by calling the `MemoryPoolMXBean.isUsageThresholdSupported()` and `MemoryPoolMXBean.isCollectionUsageThresholdSupported()` methods.

- `usage threshold exceeded notification` - for notifying that the memory usage of a memory pool is increased and has reached or exceeded its usage threshold value.
- `collection usage threshold exceeded notification` - for notifying that the memory usage of a memory pool is greater than or equal to its collection usage threshold after the Java virtual machine has expended effort in recycling unused objects in that memory pool.

The notification emitted is a `Notification` instance whose `user data` is set to a `CompositeData` that represents a `MemoryNotificationInfo` object containing information
about the memory pool when the notification was constructed. The `CompositeData` contains the attributes as described in `MemoryNotificationInfo`.

NotificationEmitter

The `MemoryMXBean` object returned by `ManagementFactory.getMemoryMXBean()` implements the `NotificationEmitter` interface that allows a listener to be registered within the `MemoryMXBean` as a notification listener. Below is an example code that registers a `MyListener` to handle notification emitted by the `MemoryMXBean`.

```
class MyListener implements javax.management.NotificationListener {
    public void handleNotification(Notification notif, Object handback) {
        // handle notification
        ....
    }
}

MemoryMXBean mbean = ManagementFactory.getMemoryMXBean();
NotificationEmitter emitter = (NotificationEmitter) mbean;
```

```
MyListener listener = new MyListener();
emitter.addNotificationListener(listener, null, null);
```

Since:

1.5

See Also:

ManagementFactory.getPlatformMXBeans(Class),
JMX Specification.,
Ways to Access MXBeans

Method Summary

All Methods | Instance Methods

Modifier and Type	Method	Description
void	**gc**()	Runs the garbage collector.
MemoryUsage	**getHeapMemoryUsage**()	Returns the current memory usage of the heap that is used for object allocation.
MemoryUsage	**getNonHeapMemoryUsage**()	Returns the current memory usage of non-heap memory that is used by the Java virtual machine.
int	**getObjectPendingFinaliza**	**Deprecated.** Finalization has been deprecated for removal.
default long	**getTotalGcCpuTime**()	Returns the approximate accumulated time, in nanoseconds, spent in garbage collection (GC).
boolean	**isVerbose**()	Tests if verbose output for the memory system is enabled.
void	**setVerbose** (boolean value)	Enables or disables verbose output for the memory system.

Methods declared in interface PlatformManagedObject

`getObjectName`

Method Details

getObjectPendingFinalizationCount

```
@Deprecated(since="18")
int getObjectPendingFinalizationCount()
```

Deprecated.
Finalization has been deprecated for removal. See `Object.finalize()` for details.

Returns the approximate number of objects for which finalization is pending.

Returns:

the approximate number objects for which finalization is pending. If this MemoryMXBean contains information about a JVM in which finalization has been disabled or removed, this method always returns zero.

getHeapMemoryUsage

`MemoryUsage getHeapMemoryUsage()`

Returns the current memory usage of the heap that is used for object allocation. The heap consists of one or more memory pools. The `used` and `committed` size of the returned memory usage is the sum of those values of all heap memory pools whereas the `init` and `max` size of the returned memory usage represents the setting of the heap memory which may not be the sum of those of all heap memory pools.

The amount of used memory in the returned memory usage is the amount of memory occupied by both live objects and garbage objects that have not been collected, if any.

MBeanServer access:
The mapped type of `MemoryUsage` is `CompositeData` with attributes as specified in `MemoryUsage`.

Returns:

a `MemoryUsage` object representing the heap memory usage.

getNonHeapMemoryUsage

`MemoryUsage getNonHeapMemoryUsage()`

Returns the current memory usage of non-heap memory that is used by the Java virtual machine. The non-heap memory consists of one or more memory pools. The `used` and `committed` size of the returned memory usage is the sum of those values of all non-heap memory pools whereas the `init` and `max` size of the returned memory usage represents the setting of the non-heap memory which may not be the sum of those of all non-heap memory pools.

MBeanServer access:
The mapped type of `MemoryUsage` is `CompositeData` with attributes as specified in `MemoryUsage`.

Returns:

a `MemoryUsage` object representing the non-heap memory usage.

getTotalGcCpuTime

```
default long getTotalGcCpuTime()
```

Returns the approximate accumulated time, in nanoseconds, spent in garbage collection (GC).

The time spent in spent in GC is the CPU time used by all GC activity, including any overhead, which means the result may be non-zero even if no GC has occurred. This method returns `-1` if the platform does not support this operation or the information is not available.

API Note:

May be used in conjunction with `jdk.management/com.sun.management.OperatingSystemMXBean#getProcessCpuTime()` for calculating the GC's usage of CPU time as a whole.

Implementation Requirements:

The default implementation returns `-1`.

Implementation Note:

The specifics on what constitutes the time spent in GC are highly implementation dependent. In the HotSpot Virtual Machine, this time includes relevant implementation-specific details such as driver threads, workers, VM Operations and string deduplication (if enabled). Driver threads may be created by a GC to orchestrate its work. The return value can be -1 if called when measurement is not possible, such as during shutdown.

Returns:

the total accumulated CPU time for GC in nanoseconds, or `-1`.

Since:

26

isVerbose

```
boolean isVerbose()
```

Tests if verbose output for the memory system is enabled.

Returns:

`true` if verbose output for the memory system is enabled; `false` otherwise.

setVerbose

```
void setVerbose(boolean value)
```

Enables or disables verbose output for the memory system. The verbose output information and the output stream to which the verbose information is emitted are implementation dependent. Typically, a Java virtual machine implementation prints a message whenever it frees memory at garbage collection.

Each invocation of this method enables or disables verbose output globally.

Parameters:

`value` - `true` to enable verbose output; `false` to disable.

gc

```
void gc()
```

Runs the garbage collector. The call `gc()` is effectively equivalent to the call:

```
System.gc()
```

See Also:

`System.gc()`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.